CSCI 6461 | TOPIC 2

RISC vs. CISC: Instruction Set Design Principles & Trade-offs

ISA boundaries, microarchitectural costs, and quantitative trade-offs

Dr. J. Burns
Computer Systems Architecture — AArch64 Reference Platform

Topic 2: Module Roadmap

1. **ISA Foundations & Complexity Distribution**
   - The programmer-visible hardware contract and historical constraints.
2. **The CISC Architectural Model**
   - Register-memory operations, variable-length encodings, and microarchitectural costs.
3. **The RISC Revolution, Principles & Comparative Metrics**
   - Load/store discipline, fixed 32-bit width, and direct architectural comparison.
4. **Quantitative Performance, Energy & Modern Convergence**
   - Iron Law evaluation, Amdahl's Law, legacy compatibility, and modern μop cracking.

PART 1

ISA Foundations & Historical Context

What Is an Instruction Set Architecture?

- An ISA is the **programmer-visible functional specification** of a processor.
- Defines instructions, register files, addressing modes, and architectural state.
- Sits directly at the hardware/software boundary as a rigid abstraction contract.
- **Complexity Redistribution:** ISA design redistributes complexity between software (compilers) and hardware (microarchitectures).

A well-designed ISA survives decades of microarchitectural overhauls by maintaining this strict software contract while hardware implementation evolves underneath it.

Historical Constraints Shaped Early ISAs (CISC Era)

- **Expensive, Slow Memory (1960s–1970s):** Small core memory and high DRAM latency placed a premium on minimizing binary program size.
- **Assembly-Centric Programming:** Programs were frequently hand-written; rich instructions helped express high-level constructs directly.
- **Microprogrammed ROM Control:** Complex multi-cycle instructions were cheap to implement via on-chip microcode routines.
- **The Semantic Gap:** Early designers believed machine instructions should mirror high-level statements directly.

The CISC Philosophy

Designers prioritized memory efficiency and expressive instructions to support human assembly programmers, willingly trading hardware decoding complexity for smaller binary sizes.

PART 2

The CISC Architectural Model

The CISC Architectural Model

- **CISC:** *Complex Instruction Set Computer* (e.g., VAX, x86).
- **Variable-Length Encodings:** Instructions range from 1 to 15 bytes to maximize code density and pack small instructions tightly.
- **Register-Memory Operations:** Arithmetic instructions can accept memory operands directly without separate load/store steps (e.g., `add [rdi], rax`).
- **Complex Addressing Modes:** Evaluates scaled indices and indirect pointers in hardware directly during execution.

The Decoding Bottleneck

Parsing variable-length instructions sequentially creates a fundamental thermal and frequency limit in modern superscalar front-ends, consuming massive transistor budgets just to find instruction boundaries.

CISC Execution: Register-Memory Translation

CISC Instruction Example (x86-64)

```
add [rdi + 8], rax
```

Architectural View (Software)

- 1 single instruction fetched.
- Compact variable encoding.
- Atomic arithmetic operates directly on main memory.

Hardware Decomposition (μops)

1. **Load:** Fetch contents of `[rdi+8]` into ALU.
2. **Add:** Sum loaded value with `rax`.
3. **Store:** Write result back to `[rdi+8]`.

One architectural instruction decomposes into multiple hardware micro-operations, completely abstracting the pipeline timing away from the compiler.

PART 3

The RISC Revolution & Principles

The 1980s Shift: Emergence of RISC

- **Empirical Workload Findings:** Patterson & Hennessy demonstrated that compiled code utilized $< 20\%$ of available CISC instruction repertoires over 90% of the time.
- **Pipeline Hazards:** Variable-length, irregular instructions created severe pipeline stalls and hazard control logic bottlenecks.
- **Compiler Advancements:** Modern optimizing compilers handle register allocation and instruction scheduling far more effectively than monolithic hardware.

The Compiler's New Role

Instead of relying on rigid hardware to interpret complex commands, RISC shifts the burden of scheduling, register allocation, and optimization entirely to the compiler.

The RISC Design Principles

- **Fixed 32-Bit Instruction Width:** Simplifies parallel instruction fetch and multi-decoder design.
- **Strict Load/Store Architecture:** ALU operations use *only* registers; memory access is restricted to explicit `ldr`/`str`.
- **Large Orthogonal Register File:** 31 general-purpose registers (AArch64 `x0`–`x30`) maintain active variables on-chip.
- **Single-Cycle Execution Goal:** Uniform pipeline stage latencies prevent instruction bubbles and hardware stalling.

The Hardware Benefit

By simplifying the logic required to decode and route instructions, processors can redirect massive chip area into larger L1 caches, more execution units, and deeper predictive pipelines.

Code Comparison: RISC vs. CISC Memory Addition

High-level statement: `c = a + b;` (operands in memory)

AArch64 RISC Assembly

`ldr x1, [x0, #0]`	load a
`ldr x2, [x0, #8]`	load b
`add x3, x1, x2`	compute sum
`str x3, [x0, #16]`	store c

x86-64 CISC Assembly

`mov rax, [rdi]`	load a
`add rax, [rdi + 8]`	load b + add
`mov [rdi + 16], rax`	store c

RISC explicitly uses 4 fixed-width instructions (16 bytes); CISC uses 3 variable-length instructions, but the actual silicon workload executed in the hardware datapath is identical.

Fetch Complexity: Concrete Byte Encodings

RISC Fixed 32-Bit Decode

- Every instruction is exactly 4 bytes.
- `add x0, x1, x2` $\implies$ `0x8B020020`
- Enables simple parallel decoders (e.g., 8-wide decode) by strictly fetching $\text{pc} + 4k$.

CISC Variable-Length Decode

- `add eax, ebx` $\implies$ `0x01D8` (2 bytes)
- `add rax, rbx` $\implies$ `0x4801D8` (3 bytes)
- `add rax, [rbx+rcx*4+0x10]` $\implies$ 8 bytes!
- Requires sequential pre-decoders just to locate instruction boundaries.

The Hardware Penalty

The computational effort of determining where one CISC instruction ends and the next begins requires massive transistor budgets, directly degrading a chip's Performance-per-Watt.

The Cost of Legacy Backwards Compatibility

- **The CISC Decoder Tax:** x86-64 carries decades of binary compatibility baggage (16-bit 8086, 32-bit i386). To execute modern 64-bit commands, it relies on **prefix bytes** (e.g., the 0x48 REX prefix) layered over legacy opcodes.
- **Sequential Choke Point:** A hardware decoder cannot know an instruction's true length until it reads these prefixes byte-by-byte in real time.
- **The AArch64 Clean Slate:** ARM deliberately dropped legacy 32-bit instruction compatibility (A32) within its new 64-bit execution state (A64).
- **The RISC Advantage:** This strict severing of legacy ties guaranteed a clean, zero-prefix 4-byte instruction width, enabling massively parallel instruction fetching without historical penalties.

ISA Comparison: AArch64 vs. x86-64

Design Metric	AArch64 (Modern RISC)	x86-64 (Modern CISC)
Instruction Width	Fixed 32-bit (4 bytes)	Variable (1 to 15 bytes)
ALU Operands	3-operand (`add d, s1, s2`)	2-operand destructive (`add dst, src`)
Memory Access	Load/Store only (`ldr`/`str`)	Register-Memory (`add rax, [rbx]`)
General Registers	31 orthogonal (`x0`–`x30`)	16 non-uniform (`rax`, `rbx`...)
Decode Logic	Direct parallel single-cycle	Sequential scan + μop cache
Design Focus	Compiler-driven scheduling	Dynamic hardware translation
Primary Benefit	Parallel decode & power efficiency	Compact binaries & backwards compatibility

Architectural Trade-Off

CISC minimizes binary size and maintains historical software compatibility through complex hardware decoders; RISC simplifies datapath logic to maximize decode throughput and energy efficiency.

PART 4

Quantitative Performance & Trade-offs

The Performance Equation (Iron Law)

CPU Execution Time Equation

$$T_{\text{CPU}} = IC \times CPI \times T_{\text{clk}} = \frac{IC \times CPI}{f_{\text{clk}}}$$

- **Instruction Count (IC):** Total executed instructions (influenced by ISA and compiler).
- **CPI:** Average clock cycles per instruction (influenced by pipeline and memory stalls).
- **Clock Period (T_{clk}):** Duration of one cycle (influenced by circuit depth and design).

The Optimization Tug-of-War

Enhancing one metric frequently degrades another. For instance, deepening a pipeline to increase f_{clk} often increases CPI due to more expensive branch misprediction penalties.

Worked Performance Comparison

Workload evaluation across two competing microarchitectures:

Machine A (RISC / AArch64)

- $IC = 1.4 \times 10^9$
- $CPI = 1.10$
- $f_{\text{clk}} = 3.0$ GHz

$$T_A = \frac{1.4 \times 10^9 \times 1.10}{3.0 \times 10^9} = \mathbf{0.513}\ \text{s}$$

Machine B (CISC / x86-64)

- $IC = 1.0 \times 10^9$ (-28% IC)
- $CPI = 1.60$ (Higher CPI)
- $f_{\text{clk}} = 3.0$ GHz

$$T_B = \frac{1.0 \times 10^9 \times 1.60}{3.0 \times 10^9} = \mathbf{0.533}\ \text{s}$$

RISC wins (Speedup $\approx 1.04\times$) because its highly predictable, lower CPI strictly compensates for executing more total operations.

Amdahl's Law and Performance Limits

Amdahl's Law Equation

$$S_{\text{overall}} = \frac{1}{(1-f) + \frac{f}{S_{\text{enhanced}}}}$$

- f: Fraction of execution time subjected to optimization.
- S_{enhanced}: Speedup factor achieved on that specific fraction.
- **Example ($f = 0.60, S_{\text{enhanced}} = 10$):**

$$S_{\text{overall}} = \frac{1}{0.40 + \frac{0.60}{10}} \approx \mathbf{2.17}\times$$

- **Application to Architecture:** You cannot optimize total CPU performance indefinitely if front-end decode bottlenecks $(1-f)$ remain sequentially limited by variable-length processing.

Energy, Thermal Limits & Performance-per-Watt

Dynamic Power Equation

$$P = C \cdot V_{\text{dd}}^2 \cdot f + P_{\text{static}}$$

- Complex decoders increase switching capacitance (C).
- Higher frequencies (f) require higher voltages (V_{dd}).

Performance-per-Watt

$$\frac{\text{Workload Throughput}}{\text{Watts}}$$

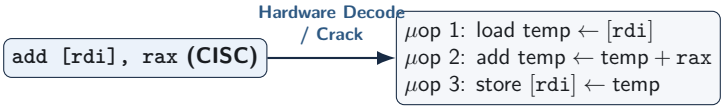
- Primary metric for mobile batteries and cloud data center racks.
- Streamlined RISC front-ends deliver high IPC at low decode energy.

The Dark Silicon Era

Because modern dense transistors cannot all be powered simultaneously without melting the chip, power efficiency is now the primary limit dictating multi-core processor performance.

Microarchitectural Convergence: Modern x86 Internals

Modern x86 processors preserve the CISC ISA externally, but execute a **RISC datapath internally**:



- The execution core is an out-of-order engine processing RISC-like micro-operations (μops).
- CISC-to-RISC translation occurs dynamically, incurring a front-end decode power overhead that RISC architectures natively bypass.

Review and Self-Test Questions

Topic 2: Comprehensive Summary

- **Complexity Redistribution:** ISA design strictly governs the workload distribution between software compilers and hardware decoders.
- **CISC Trade-offs:** Maximizes code density via variable-length encodings and register-memory math, at the steep cost of front-end decode bottlenecks and legacy prefix overhead.
- **RISC Trade-offs:** Enforces pipeline regularity via fixed 32-bit widths and strict load/store rules, dropping backwards compatibility to prioritize low CPI and high parallelism.
- **Microarchitectural Convergence:** Modern x86 fundamentally validates RISC by internally cracking CISC instructions into μops.
- **Evaluation Metrics:** True architectural efficiency is evaluated via the Iron Law ($T_{\text{CPU}} = \frac{IC \times CPI}{f_{\text{clk}}}$) and Performance-per-Watt.

Self-Test Questions: ISA Principles & Mechanics

1. **Load/Store Boundary:** Why does a strict load/store architecture simplify pipelined execution compared to an architecture permitting memory operands directly in ALU instructions?

2. **Parallel Decode Efficiency:** Why is an 8-wide instruction decoder significantly simpler and more power-efficient for a fixed 32-bit RISC ISA than for variable-length CISC?

3. **Micro-operation Translation:** When an x86 processor executes `add [rdi + 8], rax`, how does the front-end decoder functionally translate this instruction into internal μops?

Self-Test Questions: Quantitative Analysis & Power

1. **Iron Law Calculation:** A RISC core executes 1.5×10^9 instructions at CPI $= 1.1$ on a 3.0 GHz CPU. A CISC core executes 1.1×10^9 instructions at CPI $= 1.6$ on a 3.2 GHz CPU. Which finishes faster?

2. **Front-End Power Cost:** Why does instruction decoding consume a measurably higher percentage of core power in modern x86 processors than in modern AArch64 cores?

3. **Performance-per-Watt:** Why has Performance-per-Watt superseded raw clock frequency as the primary architectural design metric in modern computing?

Looking Ahead

Next Lecture Preview

Our Next Topic: The Hardware/Software Interface & AArch64 Programmer's Model

- **Architectural State:** Defining the precise software-visible hardware contract (registers, condition flags, stack pointer, and memory space).
- **AArch64 Register File:** Examining register naming conventions (`x0`–`x30` vs. `w0`–`w30`), the zero register (`xzr`), and standard ABI calling conventions.
- **Instruction Mechanics:** Exploring foundational instruction classes, memory addressing modes, and control flow translation.